Movientum: A Privacy-Preserving Federated Movie Recommendation Platform

System Architecture, Design, and Implementation Documentation

Movientum Engineering Team
Full-Stack Web Platform with Federated Contrastive Learning
System Design Documentation v1.0

***Abstract*—Movientum is a full-stack movie recommendation and discovery platform that combines a modular FastAPI backend, a React single-page frontend, PostgreSQL persistent storage, Redis caching, and a privacy-preserving federated recommendation engine based on Federated Personalized Contrastive Learning (FedPCL). This document provides comprehensive technical documentation of all system components: backend architecture, API design, database schema, authentication, search, rating system, data ingestion pipeline, recommendation evolution (rule-based to federated ML), MLOps lifecycle, containerization, API gateway design, and future scalability roadmap. The platform evolves through five implementation phases, culminating in an on-device federated training loop where user interaction data never leaves the client device, addressing both personalization quality and GDPR compliance.**

***Index Terms*—federated learning, recommendation system, FastAPI, React, PostgreSQL, Redis, Docker, LightGCN, contrastive learning, privacy-preserving ML, microservices, system architecture**

I. Introduction

Movie recommendation systems face three fundamental tensions: (1) personalization requires rich behavioral data, yet privacy regulations restrict centralized data collection; (2) cold-start users provide insufficient signal for collaborative filtering, yet rule-based fallbacks produce generic recommendations; (3) diverse user tastes (non-IID data) degrade global model quality, yet per-user models are computationally prohibitive.

Movientum addresses these tensions through a phased architecture. Phases 1–4 establish a production-quality full-stack platform with rule-based recommendations, enabling immediate deployment and real user data accumulation. Phase 5 integrates FedPCL [1], a federated recommendation algorithm that trains personalized GNN-based models locally on user devices, aggregates only privacy-protected gradient updates, and maintains cluster-level item embeddings to handle non-IID preferences.

This document serves as complete technical reference for all Movientum subsystems. Section II covers overall system architecture. Section III details the FastAPI backend. Section IV describes the database schema. Section V covers the authentication system. Section VI explains data ingestion. Section VII documents search. Section VIII covers the rating system. Section IX describes the recommendation system evolution. Section X provides the FedPCL deep dive. Section XI covers the React frontend. Section XII documents the news system. Section XIII covers containerization. Section XIV describes API gateway design. Section XV documents the ML lifecycle. Section XVI covers storage estimation. Section XVII outlines the scaling roadmap. Section XVIII documents configuration design. Section XIX presents the phased implementation plan. Section XXII concludes.

II. System Architecture

A. Architecture Philosophy

Movientum adopts a *modular monolith* strategy at launch. All backend features reside in one deployable service, but are structured with clear module boundaries — enabling clean future extraction into microservices without rewriting business logic. The frontend is a separate React SPA deployed independently behind Nginx.

B. High-Level Component View

The platform comprises six primary infrastructure layers:

1) **Client Layer**: React SPA served as static files
2) **Ingress Layer**: Nginx reverse proxy handling SSL, routing, rate limiting
3) **Application Layer**: FastAPI backend (4 Uvicorn workers)
4) **Cache Layer**: Redis for distributed key-value caching
5) **Data Layer**: PostgreSQL as primary persistent store
6) **Async Layer**: Celery worker for background and scheduled tasks

C. Request Routing

Nginx splits incoming traffic on two rules:

```
location /api/ {
  proxy_pass http://backend:8000;
}
location / {
  try_files $uri $uri/ /index.html;
}
```

API requests traverse the full middleware stack; static requests are served directly from the built React bundle, keeping backend load minimal.

D. Component Responsibility Matrix

TABLE I
COMPONENT RESPONSIBILITIES

Component	Technology	Responsibility
Nginx	Nginx 1.25	Proxy, SSL, static serve
React Frontend	React 18 + Router v6	UI rendering, auth state
FastAPI Backend	Python 3.11 + FastAPI	API, business logic
PostgreSQL	PostgreSQL 15	Primary persistent store
Redis	Redis 7	Cache, session, broker
Celery	Celery 5	Background/cron tasks
FedPCL Module	PyTorch + NumPy	Federated training coord.
TMDB API	External REST	Movie metadata source
NewsAPI	External REST	News articles source
MLflow	MLflow 2.x	Experiment tracking

E. Inter-Component Communication

Table II summarizes how components communicate.

TABLE II
INTER-COMPONENT COMMUNICATION PROTOCOLS

From	To	Protocol	Data
Browser	Nginx	HTTPS	JSON + JWT
Nginx	FastAPI	HTTP	Forwarded request
FastAPI	PostgreSQL	TCP (asyncpg)	SQL → objects
FastAPI	Redis	TCP	JSON strings
FastAPI	TMDB/News	HTTPS	REST JSON
FastAPI	Celery	Redis queue	Task messages
Celery	PostgreSQL	TCP	DB writes

F. Deployment Architecture (MVP)

All containers run on a single Ubuntu 22.04 VPS (4 vCPU, 8 GB RAM) connected via Docker bridge network `movientum_network`. Container-to-container communication uses Docker DNS (service names, not IP addresses). Only Nginx exposes ports 80/443 externally; all other containers are internal-only.

G. Deployment Architecture (Scale-Out)

When single-server capacity is exhausted (typically > 300 concurrent users), the platform transitions to:

- Multiple FastAPI instances behind a load balancer (least-connections algorithm)
- Separate managed PostgreSQL with read replicas
- Redis Cluster or Redis Sentinel
- CDN for static assets and TMDB image proxying

FastAPI instances are fully stateless: JWT carries auth state; user data lives in PostgreSQL; cache in Redis. Any instance can serve any request.

III. BACKEND SYSTEM

A. Technology Choice: FastAPI

FastAPI was selected over Django/Flask for four reasons: (1) native async support via `asyncio` and `asyncpg` handles concurrent I/O efficiently; (2) automatic OpenAPI documentation generation; (3) Pydantic models provide compile-time type safety and runtime validation; (4) fast development iteration with near-zero boilerplate.

B. Three-Layer Architecture

The backend enforces strict separation into three layers:

Layer 1 — Routers (HTTP Layer): Define URL paths and HTTP methods. Validate incoming request shapes via Pydantic. Delegate to service functions. Return HTTP responses. Routers contain *no business logic*.

Layer 2 — Services (Business Logic Layer): Core of the application. Orchestrate data flow between repositories, apply business rules, call external APIs, manage caching decisions, and dispatch background tasks. Services contain *no HTTP knowledge and no raw DB queries*.

Layer 3 — Repositories (Data Access Layer): The only layer that touches the database. Execute SQL via SQLAlchemy ORM, return domain objects. Swapping the database requires changes only here.

C. Directory Structure

```
/app
  main.py            # App entry, middleware,
      router mount
  config.py          # Pydantic Settings reads .
      env

  /routers           # HTTP layer (thin)
    auth.py  movies.py  ratings.py
    watch.py  search.py  recommendations.py
    news.py  fedpcl.py

  /services          # Business logic (thick)
    auth_service.py  movie_service.py
    rating_service.py  search_service.py
    recommendation_service.py  news_service.py

  /repositories      # DB queries only
    user_repo.py  movie_repo.py
    rating_repo.py  watch_repo.py

  /models            # Pydantic request/response
      schemas
    user_models.py  movie_models.py  rating_models
        .py

  /db
    database.py      # Async SQLAlchemy engine +
        session
    orm_models.py    # SQLAlchemy ORM table
        definitions

  /middleware
    auth_middleware.py
    error_handler.py
    logging_middleware.py

  /utils
    jwt_utils.py  password_utils.py  cache_utils.
        py
```

D. Separation of Concerns

TABLE III
LAYER SEPARATION OF CONCERNS

Layer	Knows About	Does NOT Know
Router	HTTP, URLs, request shape	DB, business rules
Service	Business rules, cache, ext. APIs	HTTP, SQL
Repository	SQL, DB structure	HTTP, business rules
Middleware	Request/response lifecycle	Business data

E. Request Lifecycle

A complete request lifecycle for `GET /api/v1/movies/123`:

1) HTTP request arrives at FastAPI
2) CORS middleware checks origin
3) Auth middleware validates JWT, attaches user to request state
4) Logging middleware records method, path, IP, timestamp
5) Router matches path, validates path param (must be integer)
6) Router calls `movie_service.get_movie_by_id(123)`
7) Service checks Redis cache key `movie:detail:123`
   - Cache HIT → return immediately (∼1 ms)
   - Cache MISS → proceed to repository
8) Repository executes `SELECT` against PostgreSQL
9) Service enriches data (e.g., user-specific watch status)
10) Service stores result in Redis (TTL: 3600s)
11) Router serializes response via Pydantic model → JSON
12) HTTP 200 response sent to client

F. API Endpoint Groups

1) Authentication Endpoints:

```
POST /api/v1/auth/register   # Create new user
POST /api/v1/auth/login      # Return JWT token
POST /api/v1/auth/logout     # Blacklist token
POST /api/v1/auth/refresh    # Refresh expired JWT
GET  /api/v1/auth/me         # Current user
    profile
```

2) Movie Endpoints:

```
GET /api/v1/movies              # Paginated/
    filtered list
GET /api/v1/movies/{id}         # Single movie
    detail
GET /api/v1/movies/trending     # Trending movies
GET /api/v1/movies/genre/{g}    # Movies by genre
```

3) Search Endpoints:

```
GET /api/v1/search?q={query}
GET /api/v1/search/autocomplete?q={query}
```

4) Rating Endpoints:

```
POST   /api/v1/ratings           # Submit rating
GET    /api/v1/ratings/me        # My ratings
GET    /api/v1/ratings/movie/{id}# Ratings for
    movie
PUT    /api/v1/ratings/{id}      # Update rating
DELETE /api/v1/ratings/{id}      # Delete rating
```

5) Watch History & Watchlist:

```
POST   /api/v1/watch             # Mark watched
GET    /api/v1/watch/history     # My history
POST   /api/v1/watch/watchlist   # Add to
    watchlist
DELETE /api/v1/watch/watchlist/{id}
GET    /api/v1/watch/watchlist
```

6) Recommendation Endpoints:

```
GET /api/v1/recommendations
GET /api/v1/recommendations/similar/{movie_id}
```

G. Middleware Stack

Middleware executes in order on every request:

1) **CORS**: Allow configured origins
2) **Rate Limiter**: Block >100 req/min per IP
3) **Request Logger**: Log method, path, IP, timestamp
4) **Auth Middleware**: Validate JWT, attach user to state
5) *Router handles request*
6) **Response Logger**: Log status code, response time
7) **Error Handler**: Catch unhandled exceptions → clean JSON error

H. Error Handling Strategy

All errors return a consistent JSON shape:

```
{
  "error": "MOVIE_NOT_FOUND",
  "message": "No movie found with id 999",
  "status_code": 404,
  "timestamp": "2024-01-01T12:00:00Z"
}
```

Custom exception hierarchy:

```
MovientumException (base)
  AuthException
    InvalidCredentialsException
    TokenExpiredException
    UnauthorizedException
  MovieNotFoundException
  RatingException
    DuplicateRatingException
  ExternalAPIException
    TMDBUnavailableException
    NewsAPIException
```

HTTP status code semantics follow RFC 7231: 200 (success), 201 (created), 400 (bad request), 401 (unauthorized), 403 (forbidden), 404 (not found), 409 (conflict), 422 (validation failure), 429 (rate limited), 500 (internal error), 503 (external API down).

I. Background Tasks

Long-running operations execute asynchronously via FastAPI `BackgroundTasks` or Celery:

- Post-rating: trigger recommendation cache invalidation
- Post-watch: update FedPCL interaction log
- Periodic (cron, 2hr): fetch news batch from NewsAPI
- Periodic (cron, 3AM daily): TMDB incremental movie sync
- Periodic (bi-weekly): initiate FedPCL training round

Background tasks do not block HTTP responses. Users receive immediate 200 OK while the backend processes asynchronously.

IV. Database System

A. Technology Choice

PostgreSQL was chosen as primary database for ACID compliance (ratings and auth data require consistency), complex join support (recommendation queries), JSONB for flexible metadata, and its mature ecosystem. Redis serves as secondary data store for caching — never as primary storage.

B. Core Table Definitions

1) Users Table: Stores all registered accounts. Uses UUID primary keys to prevent enumeration attacks. Passwords stored exclusively as bcrypt hashes.

TABLE IV
USERS TABLE SCHEMA

Column	Type	Constraint	Notes
id	UUID	PK	Auto-generated
email	VARCHAR(255)	UNIQUE, NN	Login key
username	VARCHAR(100)	UNIQUE, NN	Display name
password_hash	TEXT	NN	bcrypt only
avatar_url	TEXT	NULL	Profile image
created_at	TIMESTAMPTZ	NN, DEFAULT NOW()	
is_active	BOOLEAN	DEFAULT TRUE	Soft disable
role	VARCHAR(20)	DEFAULT 'user'	user/admin
genre_prefs	TEXT[]	NULL	Preference seed

2) Movies Table: Master movie catalog sourced from TMDB. Uses TMDB integer IDs as primary key to avoid ID mismatch on sync.

TABLE V
MOVIES TABLE SCHEMA (KEY COLUMNS)

Column	Type	Notes
id	INTEGER	PK (TMDB ID)
title	VARCHAR(500)	NN
overview	TEXT	Plot summary
release_date	DATE	
runtime	INTEGER	Minutes
poster_path	TEXT	TMDB relative path
popularity	FLOAT	TMDB score
vote_average	FLOAT	Community rating
metadata	JSONB	Flexible extra data
fetched_at	TIMESTAMPTZ	Last TMDB sync time

3) Ratings Table: Multi-dimensional categorical rating with five scored dimensions. The `overall_score` field is required; all others are optional.

TABLE VI
RATINGS TABLE SCHEMA

Column	Type	Notes
id	UUID	PK
user_id	UUID	FK → users
movie_id	INTEGER	FK → movies
story_score	FLOAT	CHECK 0–10, NULL
acting_score	FLOAT	CHECK 0–10, NULL
direction_score	FLOAT	CHECK 0–10, NULL
visuals_score	FLOAT	CHECK 0–10, NULL
overall_score	FLOAT	CHECK 0–10, NN
review_text	TEXT	Optional prose
UNIQUE	(user_id, movie_id)	One rating/user/movie

4) Watch History Table:

```
CREATE TABLE watch_history (
  id          UUID PRIMARY KEY,
  user_id     UUID NOT NULL REFERENCES users(id),
  movie_id    INTEGER NOT NULL REFERENCES movies(id
      ),
  watched_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(user_id, movie_id)
);
```

5) Junction Tables: `movie_genres(movie_id, genre_id)` and `movie_directors(movie_id, director_id)` implement the many-to-many relationships with composite primary keys.

C. Indexing Strategy

TABLE VII
CRITICAL INDEXES

Table	Index	Purpose
users	UNIQUE(email)	Login lookup
movies	(popularity DESC)	Trending query
movies	GIN(search_vector)	Full-text search
movies	(release_date)	Date range filter
ratings	(user_id)	My ratings
ratings	(movie_id)	Movie ratings
watch_history	(user_id, watched_at DESC)	Recent history

D. Full-Text Search Setup

PostgreSQL native full-text search is enabled via a `tsvector` computed column:

```
ALTER TABLE movies ADD COLUMN search_vector
    tsvector;
CREATE INDEX movies_fts_idx ON movies USING GIN(
    search_vector);
CREATE TRIGGER movies_fts_trigger
  BEFORE INSERT OR UPDATE ON movies
  FOR EACH ROW EXECUTE FUNCTION
  tsvector_update_trigger(search_vector,
    'pg_catalog.english', title, overview);
```

E. Query Optimization

Avoid N+1: All list queries use JOIN to fetch related data in one round-trip rather than looping.

Aggregation Caching: Movie average rating (`avg_rating`) is stored as a denormalized column on the `movies` table and updated via trigger on rating insert/update. No `AVG()` aggregate on every request.

Pagination: Cursor-based for large lists (`WHERE id > last_seen_id LIMIT 20`) to avoid slow offsets at depth.

F. Migration Strategy

All schema changes use **Alembic**. No manual `ALTER TABLE` in production. Each change generates a versioned migration file, committed to git, applied via `alembic upgrade head` on deploy.

V. Authentication System

A. JWT Architecture

Movientum uses stateless JWT authentication. The token payload carries:

```
{
  "sub": "user-uuid",
  "email": "user@example.com",
  "role": "user",
  "iat": 1700000000,
  "exp": 1700003600,
  "jti": "unique-token-id"
}
```

Access token TTL: 60 minutes. **Refresh token TTL:** 30 days. The `jti` field enables blacklisting on logout — the token ID is stored in Redis with TTL equal to the token's remaining lifetime.

B. Registration Flow

1) Frontend validates: email format, password $\geq$ 8 chars, passwords match
2) `POST /api/v1/auth/register` {name, email, password}
3) Backend checks email uniqueness $\rightarrow$ 409 if duplicate
4) `bcrypt.hash(password, rounds=12)` — 12 rounds chosen to balance security vs. latency ($\sim$150–300 ms, acceptable for auth-only path)
5) Insert user (UUID, email, username, hash) into DB
6) Generate access + refresh tokens
7) Return 201: {token, refresh_token, user}
8) Frontend stores tokens in `localStorage`, updates `AuthContext`
9) Redirect to `/onboarding` for genre preference selection

C. Login Flow

1) `POST /api/v1/auth/login` {email, password}
2) Rate limit check: max 5 attempts/15 min/IP $\rightarrow$ 429 if exceeded
3) `SELECT` user by email. If not found: return 401 "Invalid credentials"
4) `bcrypt.verify(submitted, stored_hash)`. No match: 401 same message
5) *Identical error prevents email enumeration attacks*
6) Check `is_active == True` $\rightarrow$ 403 if disabled
7) Generate and return tokens

D. Token Validation (Every Protected Request)

1) Extract Bearer token from `Authorization` header
2) Verify HMAC-SHA256 signature with `JWT_SECRET_KEY`
3) Check `exp` claim — 401 "Token expired" if past
4) Check `jti` in Redis blacklist — 401 "Token revoked" if present
5) Attach `request.state.user` = {`id`, `email`, `role`}

E. Token Refresh (Transparent)

When access token expires, the Axios response interceptor intercepts 401 responses automatically:

1) Detect 401 with error code `TOKEN_EXPIRED`
2) Check refresh-in-progress flag (prevents infinite loop)
3) `POST /api/v1/auth/refresh` {refresh_token}
4) Backend validates refresh token, returns new access token
5) Store new token; retry original failed request
6) User perceives only a brief delay ($\sim$200 ms)

F. Logout

1) `POST /api/v1/auth/logout` with current access token
2) Backend extracts `jti`, sets Redis key `blacklist:{jti}` with TTL = token's remaining lifetime
3) Blacklist refresh token too
4) Frontend clears `localStorage`, resets `AuthContext`, navigates to `/login`

G. Role-Based Access Control

Two roles at launch: `user` and `admin`. Admin-only routes (`/api/v1/admin/*`) check `request.state.user.role == 'admin'` via a FastAPI dependency before executing.

H. Security Practices

- HTTPS enforced; HTTP redirected (HSTS header set)
- CORS restricted to Movientum frontend domain
- All inputs sanitized (HTML stripped); SQL injection prevented by ORM parameterized queries
- Passwords never returned in any API response
- Rate limiting on auth endpoints (login: 5/15 min, register: 10/hr per IP)

VI. Data Fetch System

A. Primary Source: TMDB API

The Movie Database (TMDB) provides the backbone movie catalog: 500,000+ movies, poster/backdrop images, cast/crew, popularity scores, vote averages, similar movies, and trailer links (YouTube via TMDB videos endpoint). TMDB is free with API key, with a rate limit of $\sim$40 requests/10 seconds.

B. Data Ingestion Pipeline

1) Phase 1: One-Time Bulk Seed: Run once at platform launch to populate the empty database:

1) Fetch 25 pages of `/movie/popular` → 500 movie IDs
2) Fetch 25 pages of `/movie/top_rated` → 500 more IDs (deduplicate)
3) For each unique ID: `GET /movie/{id}` (full details) + `GET /movie/{id}/credits` (directors)
4) Insert into `movies`, `genres`, `movie_genres`, `directors`, `movie_directors`
5) Sleep 0.25s between requests (stays within rate limit)

Expected duration: 30–60 minutes. Expected result: 800–1000 unique movies.

2) Phase 2: Incremental Daily Sync: Cron job at 3 AM daily:

1) Fetch `/movie/now_playing` and `/movie/upcoming` (3 pages each)
2) Insert new movie IDs not yet in DB
3) Update `popularity` and `vote_average` for top 1000 existing movies
4) Invalidate Redis keys `movie:trending:*` and `movie:list:*`

3) Phase 3: On-Demand Fetch (Search Fallback): When a user searches and local DB returns < 5 results: call TMDB `/search/movie`, store new movies, return merged results. This lazily expands the catalog based on user interest.

C. Three-Tier Caching Strategy

TABLE VIII
CACHING TIERS AND TTLS

Tier	Store	TTL
movie:detail:{id}	Redis	3600s (1 hr)
movie:trending	Redis	1800s (30 min)
search:results:{hash}	Redis	600s (10 min)
autocomplete:{prefix}	Redis	300s (5 min)
user:recommendations:{uid}	Redis	900s (15 min)
genre_list	In-memory	86400s (24 hr)
news:feed:*	Redis	7200s (2 hr)

D. Fetch Decision Tree

For every data request the backend follows a stale-while-revalidate pattern:

1) Check Redis → HIT: return immediately
2) Check PostgreSQL: HIT + data fresh (fetched_at $<$ 24h) → load, populate Redis, return
3) HIT + stale (fetched_at $>$ 24h) → return stale immediately, trigger background TMDB refresh
4) MISS → fetch from TMDB, store in DB and Redis, return
5) TMDB unavailable → return stale DB data, log error

This ensures users always receive a fast response regardless of data freshness.

E. Image Handling

TMDB images are never stored locally. Image URLs are constructed at render time:

```
https://image.tmdb.org/t/p/{size}{poster_path}
```

Size choices: `w185` (thumbnails), `w342` (MovieCard grid), `w500` (detail page), `w780` (backdrop), `w1280` (hero banner). TMDB CDN handles global distribution and bandwidth.

VII. SEARCH SYSTEM

A. Two-Layer Architecture

Layer 1 — Local DB (Primary): PostgreSQL full-text search on `movies` table. Latency: 5–20 ms. Coverage: movies already in local DB.

Layer 2 — TMDB Fallback (Secondary): Called only when local results $<$ threshold (default 5). Latency: 200–500 ms (network). Coverage: entire TMDB catalog. New results stored in DB for future queries.

B. Full Search Flow

1) Check Redis cache for `search:results:{query_hash}`
2) Execute PostgreSQL FTS: `WHERE search_vector @@ to_tsquery('english', query)`
3) If results ≥ 5: rank, cache, return
4) Else: augment with TMDB, store new movies, merge, cache, return

C. Autocomplete Flow

1) Debounce 300ms; minimum 2 characters
2) Check Redis `autocomplete:{prefix}`
3) If miss: `SELECT id, title, release_date, poster_path FROM movies WHERE LOWER(title) LIKE '{prefix}%' LIMIT 8`
4) Cache 300s, return list with title + year + thumbnail

D. Ranking Formula

Results are ranked by composite score:

$$\text{score} = 0.50 \cdot r_{\text{relevance}} + 0.30 \cdot r_{\text{popularity}} + 0.20 \cdot r_{\text{rating}} \quad (1)$$

where $r_{\text{relevance}}$ is PostgreSQL `ts_rank`, $r_{\text{popularity}}$ is normalized TMDB popularity, and r_{rating} is normalized `vote_average`. Exact title matches always rank first. Movies $<$ 6 months old receive a 10% popularity boost.

E. Edge Case Handling

TABLE IX
SEARCH EDGE CASES

Case	Handling
Empty query	Return empty, no DB call
$<$ 2 chars	No autocomplete call
TMDB down	Return local results only
Typos (future)	pg_trgm trigram fuzzy matching
Non-English	Pass directly to TMDB

VIII. Rating System

A. Rating Categories

Movientum uses a four-category qualitative rating system rather than a numeric scale:

TABLE X
Rating Categories

Category	Color	Meaning	FedPCL Weight
Skip	#FF4D6D (Red)	Not worth time	-1.0
Timepass	#FFC300 (Yellow)	Decent, forgettable	$+0.3$
Go for it	#00E5A0 (Green)	Recommend	$+0.7$
Perfection	#9B59FF (Purple)	Masterpiece	$+1.0$

B. User Rating API

```
POST /api/v1/ratings
Authorization: Bearer <JWT>
Body: {"movie_id": "tt1234567", "category": "
    go_for_it"}

Response 200:
{
  "success": true,
  "user_rating": "go_for_it",
  "updated_distribution": {
    "skip": 1, "timepass": 11,
    "go_for_it": 80, "perfection": 9, "total": 101
  }
}
```

Implemented as UPSERT: one rating per user per movie; re-rating replaces old value.

C. Predefined (Bulk) Ratings

Administrators may import pre-classified rating data from external critic pools via:

```
POST /api/v1/admin/ratings/bulk
Body: [
  {
    "movie_id": "tt1234567",
    "source": "critics_pool_2024",
    "ratings": {
      "skip": 2, "timepass": 15,
      "go_for_it": 112, "perfection": 13
    }
  }
]
```

Idempotent by `(movie_id, source)` composite key. Combined distribution for meter display:

$$D_{\text{combined}} = D_{\text{user}} + D_{\text{predefined}} \tag{2}$$

D. Semicircular Meter UI

The rating meter renders as a 180° arc divided proportionally by category:

$$\theta_c = \frac{D_c}{\sum_c D_c} \times 180 \tag{3}$$

Center text shows dominant category percentage and total vote count. Arc segments rendered left-to-right: Skip → Timepass → Go for it → Perfection.

E. Database Schema

```
CREATE TABLE user_ratings (
  id         UUID PRIMARY KEY DEFAULT
      gen_random_uuid(),
  user_id    UUID NOT NULL REFERENCES users(id),
  movie_id   VARCHAR(20) NOT NULL,
  category   VARCHAR(20) NOT NULL
             CHECK (category IN
               ('skip','timepass','go_for_it','
                   perfection')),
  created_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE (user_id, movie_id)
);

CREATE TABLE rating_distribution_cache (
  movie_id   VARCHAR(20) PRIMARY KEY,
  skip       INT DEFAULT 0,
  timepass   INT DEFAULT 0,
  go_for_it  INT DEFAULT 0,
  perfection INT DEFAULT 0,
  total      INT DEFAULT 0,
  updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

The `rating_distribution_cache` table eliminates expensive aggregation queries. It is rebuilt on every rating insert/update via service-layer logic.

IX. Recommendation System

A. Evolution Phases

The recommendation system evolves through three phases sharing the same API endpoint contract:

TABLE XI
Recommendation System Evolution

Phase	Algorithm	Privacy	Personalization
1	Rule-based	N/A	Medium
2	Collaborative Filtering	None	Good
3	FedPCL	LDP ($\varepsilon = 100$)	Strong

All phases use identical API endpoint: `GET /api/v1/recommendations`. Backend implementation upgrades transparently without frontend changes.

B. Phase 1: Rule-Based Recommendations

For authenticated users with watch history, the blending formula is:

$$R = 0.40 \cdot R_{\text{genre}} + 0.20 \cdot R_{\text{director}} + 0.20 \cdot R_{\text{similar}} + 0.20 \cdot R_{\text{trending}} \tag{4}$$

where:

- R_{genre}: top movies in user's most-watched genres, not yet watched
- R_{director}: other films by directors the user watched ≥ 2 times
- R_{similar}: movies matching genre + vote_average of high-rated films
- R_{trending}: global TMDB trending, not yet watched

Post-process: deduplicate, filter watched, apply 10% exploration budget (random genre picks), return top 20.

C. Phase 2: Collaborative Filtering

Matrix Factorization (ALS or SGD) decomposes the user-item interaction matrix $M \in \mathbb{R}^{|U|\times|I|}$ into user factors $P \in \mathbb{R}^{|U|\times d}$ and item factors $Q \in \mathbb{R}^{|I|\times d}$:

$$\hat{r}_{ui} = p_u \cdot q_i^T \tag{5}$$

Implicit feedback weights:

TABLE XII
IMPLICIT FEEDBACK WEIGHTS

Signal	Weight
Watched	1.0
Rated ≥ 7.0	2.0
Rated < 5.0	-0.5
Added to watchlist	0.5
Detail page viewed	0.3

D. Cold Start Mitigation

- Onboarding survey: 5 genre questions at registration
- Show top movies in selected genres until 10+ interactions accumulated
- Explicit ratings prompt: ask users to rate films they have already seen
- Popularity fallback for new movies (no ratings yet)

E. Recommendation Diversity

Pure ML risks filter bubbles. Diversity strategies:

- 10% exploration budget (random genre picks)
- Minimum 3 genres in top-20 results
- 10% recency bonus for movies < 6 months old

X. FEDPCL: FEDERATED PERSONALIZED CONTRASTIVE LEARNING

A. Problem Formulation

FedPCL [1] addresses three fundamental challenges in federated recommendation:

TABLE XIII
FEDPCL PROBLEM-SOLUTION MAPPING

Problem	Cause	Solution
Data sparsity	$\sim$20–100 interactions/user	2-hop structural CL
Non-IID data	Heterogeneous tastes	K-means cluster models
Privacy	Gradient leakage	Local Differential Privacy

B. User-Item Interaction Graph

Each user's interactions form a bipartite graph $G = (U \cup I, E)$ where U is the user set, I the item set, and edges E represent watch/rating interactions. In Movientum:

- Nodes: users (registered accounts) + movies (TMDB catalog)
- Edges: `watch_history` records + `ratings` records
- Graph is locally stored per user — never assembled globally

1-hop neighbors: movies user has directly interacted with.

2-hop neighbors: users who interacted with the same movies (structural similarity signal without raw data sharing).

C. LightGCN Embedding Generation

Movientum uses LightGCN [2] as the GNN backbone. Standard GCN message passing is simplified by removing weight matrices and activation functions:

Layer 0 (Initialization):

$$e_u^{(0)} \in \mathbb{R}^d \quad \text{(user embedding, random init)} \tag{6}$$

$$E_i^{(0)} \in \mathbb{R}^d \quad \text{(item embedding, from cluster model)} \tag{7}$$

Layer 1 (1-hop aggregation):

$$e_u^{(1)} = \frac{1}{\sqrt{|N_u|}} \sum_{i \in N_u} E_i^{(0)} \tag{8}$$

$$E_i^{(1)} = w_a[i] \cdot e_u^{(0)} + \sum_{v \in N_i} W_{nv}[i, v] \cdot e_v^{(0)} \tag{9}$$

Layer 2 (2-hop aggregation):

$$e_u^{(2)} = \frac{1}{\sqrt{|N_u|}} \sum_{i \in N_u} E_i^{(1)} \tag{10}$$

Final representation (mean pooling across layers):

$$e_u^{\text{agg}} = \frac{1}{L+1} \sum_{l=0}^{L} e_u^{(l)} \tag{11}$$

Precomputed weights:

$$w_a[i] = \frac{1}{\sqrt{\deg(i)}} \tag{12}$$

$$W_{nv}[i, v] = \frac{1}{\sqrt{\deg(i) \cdot \deg(v)}} \tag{13}$$

These are computed once at initialization, not re-computed each epoch.

D. BPR Loss

Bayesian Personalized Ranking loss for pairwise recommendation:

$$\mathcal{L}_{\text{BPR}} = -\sum_{(u,i,j)\in D} \log \sigma\left(e_u^{\text{agg}} \cdot E_i^{\text{personal}} - e_u^{\text{agg}} \cdot E_j^{\text{personal}}\right) \tag{14}$$

where i is a positive item (watched/rated), j is a sampled negative item (not interacted with), and D is the set of training triples. 1 negative is sampled per positive.

E. Contrastive Learning Losses

Contrastive losses activate after `warmup_rounds=20` to allow BPR to first build stable base embeddings.

User Contrastive Loss $\mathcal{L}_{\text{Con}}^U$ (Eq. 5 in [1]):

$$\mathcal{L}_{\text{Con}}^U = -\log \frac{\exp(e_u^{(L)} \cdot e_u^{(0)}/\tau)}{\exp(e_u^{(L)} \cdot e_u^{(0)}/\tau) + \sum_{v \in \mathcal{N}_u} \exp(e_u^{(L)} \cdot e_v^{(0)}/\tau)} \tag{15}$$

Intuition: The anchor user's even-layer embedding should be close to its own layer-0 embedding (self-consistency) but

far from 2-hop neighbors (preserve uniqueness despite shared movies).

Item Contrastive Loss $\mathcal{L}_{\text{Con}}^{V}$ (Eq. 6 in [1]):

$$\mathcal{L}_{\text{Con}}^{V} = \text{CrossEntropy}\left(\frac{E^{(L)} \cdot (E^{(0)})^{T}}{\tau}\right) \tag{16}$$

where the diagonal of the $n \times n$ cosine similarity matrix represents correct self-pairs.

Total Loss:

$$\mathcal{L} = \mathcal{L}_{\text{BPR}} + \mathcal{L}_{\text{reg}} + \beta_1 \cdot \left(\mathcal{L}_{\text{Con}}^{U} + \lambda \cdot \mathcal{L}_{\text{Con}}^{V}\right) \tag{17}$$

with $\beta_1 = 0.1$, $\lambda = 1.0$, $\tau = 0.2$.

F. Non-IID Handling: K-means Cluster Models

The server maintains $K = 5$ cluster-specific item embedding tables:

$$E_u^{\text{personal}} = \mu_1 \cdot E_{k(u)}^{\text{cluster}} + \mu_2 \cdot E^{\text{global}} \tag{18}$$

with $\mu_1 = \mu_2 = 0.5$ (equal blend). K-means is re-run every 10 rounds on the uploaded (noisy) user embeddings:

$$k(u) = \arg\min_{k} \|e_u^{\text{noisy}} - c_k\|^2 \tag{19}$$

This produces taste archetypes (e.g., action fans, drama fans, arthouse fans) without requiring explicit labeling.

G. Local Differential Privacy

Applied client-side *before* any data leaves the device:

1) Clip gradients: $g_c = \text{clamp}(g, -\sigma, +\sigma)$ with $\sigma = 0.1$
2) Add Laplacian noise: $g_{\text{private}} = g_c + \text{Lap}(0, \lambda)$ with $\lambda = 0.001$
3) Privacy budget: $\varepsilon = \sigma/\lambda = 100$

TABLE XIV
LDP PRIVACY-ACCURACY TRADE-OFF

ε	**Privacy**	**Accuracy**
100	Loose (fast convergence)	Near-centralized
10	Moderate	Slight drop
1	Strong	Noticeable drop
0.1	Very strong	Significant drop

H. Client Training Procedure (Per Round)

1) Receive E^{personal}, `neigh_embs`, and config from server
2) Initialize local item embeddings from E^{personal}
3) For each local epoch $e = 1 \dots E$ (where $E = 10$):
   a. Sample 1 negative item per positive
   b. Run 2-layer LightGCN on expanded subgraph
   c. Compute e_u^{agg}
   d. Compute $\mathcal{L}_{\text{BPR}}$; if round > 20 add contrastive terms
   e. Backward pass: SGD on items ($lr = 0.1$), Adam on user ($lr = 0.001$)
4) Compute item deltas: $\Delta_i = E_i^{\text{local}} - E_i^{\text{personal}}$
5) Apply LDP: clip + add Laplacian noise
6) Upload: $\{\Delta_i^{\text{noisy}}, e_u^{\text{noisy}}, m_u\}$ where $m_u = |\text{train\_items}|$

Raw interaction data never leaves the client device.

I. Server Aggregation (FedAvg)

Global update (weighted by dataset size):

$$E_i^{\text{global}} \mathrel{+}= \frac{\sum_u m_u \cdot \Delta_i^u}{\sum_u m_u} \tag{20}$$

Cluster update (only from clients in cluster k):

$$E_i^{\text{cluster}(k)} \mathrel{+}= \frac{\sum_{u \in \mathcal{C}_k} m_u \cdot \Delta_i^u}{\sum_{u \in \mathcal{C}_k} m_u} \tag{21}$$

J. Hyperparameter Summary

TABLE XV
FEDPCL HYPERPARAMETERS (MOVIENTUM DEFAULTS)

Parameter	**Value**	**Rationale**
d (embed dim)	64	Quality vs. storage balance
L (GNN layers)	2	2-hop sufficient for CF signal
T (rounds)	400	Convergence by ∼200 rounds
N (clients/round)	128	Paper default
E (local epochs)	10	No divergence
K (clusters)	5	Paper default
μ_1, μ_2	0.5, 0.5	Equal cluster/global blend
warmup	20	BPR stabilizes first
β_1	0.1	CL loss weight
λ	1.0	Item CL weight
τ	0.2	InfoNCE temperature
lr_{item}	0.1	Item embedding SGD
lr_{user}	0.001	User embedding Adam
σ (clip)	0.1	LDP clip bound
λ_L (noise)	0.001	LDP Laplacian scale
ε	100	Privacy budget

K. Evaluation Metrics and Targets

Leave-one-out evaluation: train on all interactions except the most recent; test item ranked against 100 random negatives.

TABLE XVI
FEDPCL BENCHMARK RESULTS (FROM [1])

Dataset	**HR@10**	**NDCG@10**
Steam	80.36%	65.55%
ML-100K	63.81%	45.03%
ML-1M	62.86%	44.12%
Amazon	34.04%	22.93%
Movientum target	≥60%	≥40%

L. FedPCL vs. Baseline Comparison

TABLE XVII
SYSTEM COMPARISON

Aspect	**Centralized CF**	**FedAvg**	**FedPCL**
Privacy	None	Partial	Strong (LDP)
Non-IID	Good	Poor	Strong
Sparsity	Poor	Poor	Strong (2-hop)
GDPR	Difficult	Moderate	Easiest
Accuracy	Highest	Lower	Near-central

M. FedPCL Client-Server API

```
GET  /api/v1/fedpcl/round/status
  -> {round_id, is_active, config}

GET  /api/v1/fedpcl/model/latest
  -> {version, E_personal_b64, neigh_embs}

POST /api/v1/fedpcl/update
  Body: {
    round_id: "round_042",
    item_deltas: {movie_id: [64 floats]},
    user_emb: [64 floats],
    m_u: 47
  }
  -> {status: "received", next_round_in_days: 14}
```

XI. Frontend System

A. Architecture Philosophy

React handles UI rendering only. Business logic stays in the backend. The frontend's responsibilities are: render data, capture user input, call APIs via the service layer, and update UI based on responses. No business rules or raw SQL exist in the frontend.

B. Page Structure

TABLE XVIII
Frontend Pages

Route	Page	Auth
/	Home	Public
/login	Login	Redirect if logged in
/register	Register	Redirect if logged in
/movies	MovieList	Public
/movies/:id	MovieDetail	Public
/search	SearchResults	Public
/dashboard	Dashboard	Protected

C. State Management

React Context API handles global state at MVP. Three context slices:

- **AuthContext**: {`user, token, isLoggedIn`}; methods: login, logout, register, refreshToken
- **MovieContext**: {`currentMovie, watchHistory, watchlist`}; methods: addToWatchlist, markWatched
- **SearchContext**: {`query, results, isLoading`}

Redux Toolkit is the planned migration target when FedPCL background updates require more complex state coordination.

D. API Service Layer

All API calls route through a centralized Axios instance (`src/utils/api.js`):

```
// Request interceptor
config.headers.Authorization =
  `Bearer ${localStorage.getItem('access_token')
      }`;

// Response interceptor
if (error.response?.status === 401) {
  await refreshToken();
  return retryOriginalRequest();
}
```

Service files: `authService`, `movieService`, `ratingService`, `watchService`, `recommendService`, `newsService`. Components never call `fetch` directly.

E. Key Components

MovieCard: Core reusable component used in 5+ locations. Props: {`movie: {id, title, poster_path, year, vote_average}`}. Renders poster (TMDB URL), title, year, rating badge. Click navigates to `/movies/{id}`. Fallback: gradient placeholder if `poster_path` is null.

SearchBar: Lives in Navbar. 300ms debounce; minimum 2-character threshold. Autocomplete dropdown with up to 8 results. Click → navigate to movie detail; Enter → navigate to search results.

RatingModal: Category-based rating with four colored pill buttons. Optimistic UI update on submit; revert on API error.

F. Performance Strategies

- Page-level lazy loading via `React.lazy + Suspense`
- Image lazy loading (viewport-based)
- Skeleton screens while data fetches (not spinners)
- `useMemo`/`useCallback` for expensive computations
- Session-level client cache: fetched movie data stored in Context to avoid refetch on same-session revisit

G. Data Flow Example: Movie Detail Page

1) User clicks MovieCard → React Router navigates to `/movies/123`
2) `MovieDetailPage` mounts; three parallel API calls fire:
   - `GET /api/v1/movies/123`
   - `GET /api/v1/recommendations/similar/123`
   - `GET /api/v1/news/movie/123`
   - `GET /api/v1/watch/status/123` (if logged in)
3) Components render as each response arrives (no waterfall)
4) User clicks "Mark as Watched" → `POST /api/v1/watch`
5) Button updates to green checkmark; background task invalidates recommendation cache
6) User opens RatingModal, submits rating → `POST /api/v1/ratings`
7) Background task invalidates `user:recommendations:{user_id}`

XII. News System

A. Data Sources

Primary: NewsAPI. Free tier provides 100 requests/day, searching 30,000+ sources by keyword. Returns: title, description, URL, image, `published_at`, source name.

Secondary: TMDB Reviews. `/movie/{id}/reviews` for movie-specific review content.

Future: RSS Feeds. Variety, Hollywood Reporter, IndieWire, Collider.

B. Fetching Pipeline

Global news: Celery cron every 2 hours.

1) `GET /everything?q=movies+OR+cinema+OR+film&pageSize=50`
2) Filter: must have title, description, URL, image; not duplicate (SHA-256 URL hash check); published < 48 hours ago
3) Insert new articles into `news_articles` table
4) Extract movie title mentions $\rightarrow$ populate `news_movie_links`
5) Invalidate Redis `news:feed:*`

Movie-specific news: on-demand when user views Movie Detail Page. Cache 6 hours.

C. Personalization Scoring

$$s_{\text{news}}(a,u) = 0.50 \cdot f_{\text{genre}} + 0.30 \cdot f_{\text{watched}} + 0.15 \cdot f_{\text{director}} + 0.05 \cdot f_{\text{recency}} \quad (22)$$

where f_{genre} is genre-tag overlap with user's affinity profile (shared with recommendation service), f_{watched} is whether article mentions a watched movie, f_{director} is director match, and f_{recency} is age-based decay. FedPCL embedding updates automatically improve news personalization as the same preference signals drive both systems.

D. Deduplication

Same story from multiple outlets is deduplicated by: (1) SHA-256 URL hash for exact duplicates, (2) Jaccard title similarity > 0.8 treated as duplicate. Authority ranking for display: Variety/Deadline (Tier 1) $\rightarrow$ IGN/Collider (Tier 2) $\rightarrow$ others.

XIII. Containerization (Docker)

A. Container Architecture

All services run in Docker containers connected via `movientum_network` (bridge network). Container DNS resolution enables service-name addressing.

B. Container Designs

Frontend Container: Multi-stage build — Stage 1: `node:20-alpine` installs deps, builds React bundle. Stage 2: `nginx:alpine` serves static output. Final image $\approx$ 15 MB vs. 500 MB single-stage.

Backend Container: `python:3.11-slim`, runs `uvicorn app.main:app --workers 4`. Key dependencies: fastapi, asyncpg, sqlalchemy, redis, pydantic, python-jose, passlib[bcrypt], celery, httpx, torch, numpy, scikit-learn, mlflow.

PostgreSQL: `postgres:15-alpine`, named volume `postgres_data`. Tuning: `max_connections=100`, `shared_buffers=256MB`.

Redis: `redis:7-alpine`, `maxmemory 512mb`, `maxmemory-policy allkeys-lru`.

Celery Worker: Same image as backend, different entrypoint: `celery -A app.celery worker --concurrency=2`.

C. Docker Compose (Development vs. Production)

Key differences:

TABLE XIX
Dev vs. Production Compose Differences

Aspect	Development	Production
Images	Build from source	Pre-built from registry
Code mount	`volumes: ./backend:/app`	None
Uvicorn	`--reload`	`--workers 4`
Port 5432	Exposed to host	Internal only
Restart	Not set	`always`
MLflow	Running on port 5000	Separate concern

D. CI/CD Pipeline

On every `git push` to `main`:

1) Run `pytest` (backend) + `jest` (frontend)
2) Build Docker images, tag with git SHA
3) Push to container registry
4) SSH to production server
5) `docker compose pull` + rolling restart (`--no-deps`)
6) Run `alembic upgrade head`
7) Verify `GET /api/health` returns 200
8) Auto-rollback to previous image tag if health check fails

XIV. API Gateway Design

A. Gateway vs. FastAPI Router

TABLE XX
API Gateway vs. FastAPI Router

Aspect	FastAPI Router	API Gateway
Location	Inside application code	External infrastructure
Language	Python	Nginx config / Kong DSL
Handles	Request parsing, business logic	SSL, rate limits, routing
Scope	Per-service	Platform-wide

B. Phase 1: Nginx as API Gateway (MVP)

Nginx handles SSL termination (Let's Encrypt), HTTP$\rightarrow$HTTPS redirect, request routing, basic rate limiting, and security headers.

Rate limits:

```
Login:    5 req/min/IP
Register: 10 req/hr/IP
Global:   200 req/min/IP
```

Security headers added to every response: `Strict-Transport-Security`, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Content-Security-Policy`, `Referrer-Policy`.

Load balancing (multiple backend instances):

```
upstream backend_pool {
  least_conn;
  server backend1:8000 weight=1;
  server backend2:8000 weight=1;
  keepalive 32;
}
```

C. Phase 2: Kong Gateway (Scale)

Kong replaces Nginx as the API gateway, adding:

- **Route-level rate limiting**: strict (5/min) on login, generous (1000/min) on movie browse
- **JWT validation at gateway**: invalid tokens rejected before reaching FastAPI; valid tokens passed as `X-User-ID` header
- **Circuit breaker**: open if error rate $> 50\%$ for 30s $\rightarrow$ return 503 immediately
- **Request transformation**: inject `X-Request-ID`, strip `Authorization` header
- **Prometheus metrics**: per-route latency P50/P95/P99, error rate, rate limit hits

D. Future Microservices Routing

When Movientum splits into microservices, the gateway routes by path prefix:

```
/api/v1/auth/*          -> auth-service:8001
/api/v1/movies/*        -> movie-service:8002
/api/v1/search/*        -> search-service:8003
/api/v1/recommendations -> rec-service:8004
/api/v1/fedpcl/*        -> fedpcl-service:8005
/api/v1/news/*          -> news-service:8006
```

Frontend code never changes — same API paths always. Services deploy, scale, and update independently.

XV. MLOps Lifecycle

A. Experiment Tracking with MLflow

Every training run logs to MLflow:

Parameters: embed_dim, n_rounds, clients_per_round, local_epochs, n_clusters, μ_1, μ_2, β_1, τ, learning rates, LDP parameters.

Metrics per round: train_loss_bpr, train_loss_cl, train_loss_total, eval_hr10, eval_ndcg10, n_clients_participated.

Artifacts: E^{global} checkpoints (.npy), cluster assignments (.json), training config, eval results.

B. Model Registry

TABLE XXI
MLflow Model Registry Stages

Stage	Meaning
None	Experimental, never deployed
Staging	Candidate under evaluation
Production	Live model serving requests
Archived	Older version, available for rollback

C. Automated Retraining Pipeline

Triggers: (1) scheduled bi-weekly cron, (2) HR@10 < 0.50 (drift), (3) user base grows 20%+ since last training.

Pipeline steps:

1) **ETL**: Extract from `watch_history` + `ratings`, assign implicit feedback weights, filter eligible users ($\geq$ 10 interactions), build `train_dict` and `item2users` inverted index
2) **Validation**: Check min 100 eligible users; no null IDs; schema valid
3) **Training**: FedPCL 400 rounds with warmstart from current E^{global}
4) **Evaluation**: HR@10 and NDCG@10 on held-out 10% users
5) **Validation Gate**: HR@10 $\geq$ 0.60 AND NDCG@10 $\geq$ 0.40 AND Δ vs. production $\geq$ -0.02
6) **Deployment**: Load new E^{global} and E^{cluster} into memory, atomic swap, flush Redis recommendation caches

D. Shadow Testing

Before full promotion, route 10% of recommendation traffic to the new model for 24 hours. Auto-promote if shadow CTR within 5% of control; auto-rollback if engagement drops $>$ 10%.

E. Hot Reload Without Downtime

1) Load new embeddings into secondary memory slot
2) Atomic pointer swap: primary $\rightarrow$ secondary
3) Release old model memory
4) Flush Redis recommendation caches

No server restart required. Zero downtime.

F. Model Drift Detection

Data drift: monthly monitoring of genre distribution of new watches. Alert if shift $> 15\%$ from training distribution.

Model drift: weekly HR@10 on rolling held-out test set. Alert if < 0.55.

Embedding stability: per-round L2 norm of E^{global} and gradient norms. Alert if gradient norm $> 10\times$ previous round.

G. Business Metric Targets

TABLE XXII
Production ML Metric Targets

Metric	Definition	Target	Alert
CTR	Rec clicks / recs shown	$\geq$8%	$<$5%
Watch Completion	Watched / recommended	$\geq$30%	$<$20%
Session Length	Avg min/session	$\geq$15 min	$<$10 min
Rec $\rightarrow$ Rating	Recs that get rated	$\geq$15%	$<$8%
Discovery Rate	Recs outside pref. genres	10–20%	$>$30%

XVI. Storage Estimation

A. Methodology

Storage is estimated per table using average row sizes, then summed with index overhead and PostgreSQL system overhead. MLflow training artifacts are stored in a separate filesystem/S3 volume and excluded from the database budget.

B. Phase 1 Estimates (10,000 Users, 100,000 Movies)

TABLE XXIII
PHASE 1 STORAGE BREAKDOWN

Component	Size
movies table (∼1 KB/row)	100 MB
movie_genres + movie_directors	5 MB
directors table	3 MB
users table (∼0.3 KB/row)	3 MB
ratings (∼132 B/row, 200k rows)	32 MB
watch_history (∼105 B/row, 500k rows)	53 MB
watchlist (∼90 B/row, 150k rows)	14 MB
news_articles (rolling 7-day, 5k rows)	4 MB
All indexes	31 MB
PostgreSQL overhead	50 MB
Phase 1 Total	**∼295 MB**

C. Phase 2 Estimates (After FedPCL Integration)

$$E^{\text{global}}: \quad 100{,}000 \times 64 \times 4 \text{ B} = 25.6 \text{ MB/version} \tag{23}$$

$$E^{\text{cluster}} \times K: \quad 5 \times 25.6 \text{ MB} = 128 \text{ MB/version} \tag{24}$$

$$3 \text{ versions each}: \quad \approx 77 + 384 = 461 \text{ MB} \tag{25}$$

TABLE XXIV
PHASE 2 STORAGE BREAKDOWN

Component	Size
Phase 1 total	295 MB
FedPCL E^{global} (3 versions)	77 MB
FedPCL E^{cluster} (3 versions)	384 MB
User embeddings (serving)	3 MB
Cluster assignments	1 MB
User events log	60 MB
Phase 2 Total (DB)	**∼820 MB**

D. Growth Projections

TABLE XXV
STORAGE VS. USER GROWTH

Users	Movies	DB Storage
10,000	100,000	∼820 MB
50,000	100,000	∼2.5 GB
100,000	200,000	∼5.2 GB
200,000	200,000	∼8.5 GB
500,000	500,000	∼20 GB

At 100,000 users the platform approaches the 5–10 GB storage target and should migrate to a managed database plan with 50 GB allocation.

E. Storage Efficiency Strategies

- Never store poster/backdrop images locally — link TMDB CDN directly
- Store only news article title + description + URL, not full body text
- Limit catalog to searched/popular movies, not all 500k TMDB entries
- Archive watch history events $>$ 1 year to cold storage CSV
- Store FedPCL models in float16 to halve embedding storage

XVII. SCALABILITY AND FUTURE ROADMAP

A. Scaling Principles

1) Measure before scaling — add metrics first
2) Cache first — most read performance problems solved with Redis before scaling compute
3) Scale out not up — horizontal instances preferred over larger machines
4) Database is usually the bottleneck — protect with pooling, replicas, caching

B. MVP Capacity (Single Server)

Estimated single-server limits: 200–500 concurrent users, 100–200 RPS, ∼80% cache hit rate. Scale-out begins when concurrent users regularly exceed 300.

C. Scaling Layers

Layer 1 — Cache Improvements: Increase TTLs for stable data (genre list: forever, top-1000 movies: 6 hours). Pre-warm caches at startup. Enable client-side `Cache-Control` headers for public movie data.

Layer 2 — Backend Horizontal Scaling: Multiple FastAPI instances behind Nginx/ALB load balancer. Instances are fully stateless (JWT, PostgreSQL, Redis). Auto-scale rules: out if CPU $>$ 70% for 5 min or P95 latency $>$ 800 ms; in if CPU $<$ 30% for 10 min.

Layer 3 — Database Optimization: PgBouncer connection pooling. Read replicas for SELECT queries (auth/writes stay on primary). Horizontal partitioning (sharding) only at Netflix-scale.

Layer 4 — Redis Cluster: Upgrade when memory $>$ 10 GB or throughput $>$ 100k ops/sec.

D. Microservices Migration Path

Extraction order by business value:

1) Recommendation Service (GPU-heavy ML inference)
2) Search Service (add Elasticsearch for fuzzy/multilingual)
3) News Service (separate I/O-heavy cron jobs)
4) Auth Service (compliance isolation)

E. Future ML Improvements

Short term (3–6 months): Collaborative filtering Phase 2; implicit feedback signals (page view time, trailer plays).

Medium term (6–12 months): Full FedPCL integration; multi-modal recommendations (poster visual features); session-based real-time recommendations.

Long term (1+ year): LLM natural language queries ("find me a sad indie film from the 90s"); graph neural networks over full actor/director/movie graph; cross-domain recommendations (book $\rightarrow$ movie adaptation).

F. Real-Time Features

WebSocket integration planned for: FedPCL round status updates, taste-match notifications, live trending updates (Redis rolling-window counters pushed to connected clients every 30s).

G. Cost Tiers

TABLE XXVI
INFRASTRUCTURE COST BY STAGE

Stage	Users	Setup	Monthly
MVP	<1k	1 VPS, monolith	$30–80
Growth	1k–10k	+ managed Redis	$80–150
Scale 1	10k–50k	2–3 backends, LB	$150–400
Scale 2	50k–200k	PgBouncer, replicas	$400–1000
Scale 3	200k–1M	Kong, microservices	$1000–3000
Large	1M+	Multi-region, K8s	$3000+

XVIII. CONFIGURATION DESIGN

A. Two-File Configuration Pattern

Movientum separates runtime configuration (`params.yaml`) from system constants (`definitions.yaml`), following ML pipeline tooling conventions (Kubeflow, DVC, MLflow Projects).

TABLE XXVII
CONFIGURATION FILE RESPONSIBILITIES

File	Changes	Contents
params.yaml	Frequently	Tunable values, TTLs, hyperparams
definitions.yaml	Rarely	Business vocabulary, categories

B. params.yaml Key Sections

```yaml
cache:
  movie_detail_ttl: 3600
  trending_ttl: 1800
  user_recommendations_ttl: 900

auth:
  access_token_expiry_minutes: 60
  bcrypt_rounds: 12
  rate_limit_login_attempts: 5

fedpcl:
  embed_dim: 64
  n_rounds: 400
  clients_per_round: 128
  n_clusters: 5
  mu1: 0.5
  mu2: 0.5
  warmup_rounds: 20
  beta1: 0.1
  tau: 0.2
  clip_sigma: 0.1
  lambda_laplace: 0.001

model_validation:
  min_hr10: 0.60
  min_ndcg10: 0.40
  shadow_test_traffic_fraction: 0.10
  auto_rollback_engagement_drop: 0.10
```

C. definitions.yaml Key Sections

```yaml
rating_categories:
  - key: skip
    label: "Skip"
    fedpcl_weight: -1.0
  - key: timepass
    label: "Timepass"
    fedpcl_weight: 0.3
  - key: go_for_it
    label: "Go for it"
    fedpcl_weight: 0.7
  - key: perfection
    label: "Perfection"
    fedpcl_weight: 1.0

image_sizes:
  poster:
    card: "w342"
    detail: "w500"
  backdrop:
    hero: "w1280"
```

D. Kubeflow Integration

`params.yaml` is the DVC params file and Kubeflow pipeline parameter source. Different experiment runs use different parameter values, fully tracked in MLflow:

```
kfp run submit --pipeline fedpcl_pipeline \
  --param fedpcl.n_clusters=7 \
  --param fedpcl.embed_dim=128
```

XIX. IMPLEMENTATION PHASES

A. Phase 1: Data Fetch Setup

Goal: populate DB with movie data from TMDB so frontend has content to display.

Key tasks: TMDB API key, Alembic migrations for movie tables, `tmdb_service.py` with rate-limited HTTP calls, one-time seed script (`scripts/seed_movies.py`), Redis cache utilities, daily sync Celery cron.

Deliverables: 800+ movies in DB; Redis cache functional; `GET /api/v1/movies` returns paginated list.

B. Phase 2: Frontend System

Goal: deployable React UI showing movies, auth flow, search, and placeholder recommendations.

Key tasks: project structure with atomic component design, Axios instance with JWT interceptor, AuthContext, React Router with protected routes, all 6 pages, skeleton loading states, responsive breakpoints.

Deliverables: Full UI with real backend data; auth flow complete; `npm run build` passes.

C. Phase 3: Backend System

Goal: full FastAPI backend with all endpoints, JWT auth, Redis cache, clean modular structure.

Key tasks: all routers/services/repositories, async SQLAlchemy setup, PostgreSQL FTS index, Celery tasks, centralized error handler.

Note: `GET /api/v1/recommendations` serves rule-based logic per Equation 4. The endpoint contract is fixed — Phase 5 upgrades the backend logic without API changes.

Deliverables: All endpoints documented in Swagger (`/docs`); cache HITs visible in logs; Celery crons running.

D. Phase 4: Deployment Setup

Goal: platform running on server, accessible via domain, with HTTPS.

Key tasks: Dockerfiles for all services, Nginx config with SSL (Let's Encrypt), production Docker Compose, GitHub Actions CI/CD pipeline, health checks, UptimeRobot monitoring.

Deliverables: `https://movientum.com` live; push-to-deploy working; rollback tested.

E. Phase 5: ML & FedPCL Integration (Future)

Do not implement in Phases 1–4. Architecture already supports this; no earlier code requires changes.

Sub-phases:

1) **5a**: Offline collaborative filtering (matrix factorization) trained on accumulated interaction data
2) **5b**: Full FedPCL integration per Section X

Prerequisites satisfied after Phase 4:

- User interaction data accumulating (watch history, ratings)
- DB schema supports all ML input tables
- FedPCL router stubbed (returns 501 until Phase 5)
- MLflow can be added as new Docker service
- `params.yaml` already defines all FedPCL hyperparameters

XX. System Integration Summary

A. Complete Integration Map

The full data flow across all components:

1) **Frontend** → **Backend**: HTTPS + JWT; JSON request/response; consistent {`"data": ..., "meta": ...`} or {`"error": ...`} shape
2) **Backend** → **PostgreSQL**: async SQLAlchemy ORM; parameterized queries; Alembic migrations
3) **Backend** → **Redis**: GET/SETEX/DEL with structured key hierarchy; stale-while-revalidate pattern
4) **Backend** → **TMDB/NewsAPI**: HTTPS REST; exponential backoff on 429; graceful degradation on 503
5) **Backend** → **Celery**: Redis broker (DB 1); fire-and-forget for non-blocking operations
6) **FedPCL Server** ↔ **Client**: HTTPS REST; Base64 float32 arrays for embedding transfer
7) **Recommendations** → **News**: shared `user_genre_affinities(user_id)` function; FedPCL updates both
8) **FastAPI** → **MLflow**: HTTP logging; per-round metrics; artifact storage
9) **MLflow** → **Grafana**: PostgreSQL metrics query; alerting via Alertmanager → Slack/email

B. Request Latency Expectations

TABLE XXVIII
Expected Request Latencies

Request Type	Cache State	Latency
Movie detail	Redis HIT	<10 ms
Movie detail	Redis MISS, DB hit	50–150 ms
Search (local FTS)	Redis MISS	50–200 ms
Search + TMDB fallback	Full miss	300–800 ms
Recommendations	Redis HIT	<10 ms
Recommendations	Redis MISS	100–300 ms
Auth login	Always DB	150–300 ms (bcrypt)
Autocomplete	Redis HIT	<5 ms

XXI. Complete User Workflow Reference

A. Signup Flow

Registration → bcrypt hash → DB insert → JWT generation → onboarding genre selection → Home with seeded recommendations. Duplicate email returns 409. Same error message for invalid email vs. wrong password prevents enumeration.

B. Login Flow

Form → rate limit check → bcrypt verify → token generation → redirect to `?redirect` param or Home. Token refresh is transparent: Axios interceptor catches 401, refreshes, retries original request.

C. Browse → Watch → Rate

Home page fires three parallel API calls (trending, recommendations, genre rows). MovieDetail fires four parallel calls (detail, similar, news, watch status). Watch and rate actions fire background tasks to invalidate recommendation cache. FedPCL local training data is updated client-side.

D. Search → Autocomplete → Click

300ms debounce → autocomplete API → dropdown. Click on suggestion navigates directly to movie detail (skipping search results page). Enter navigates to full search results with filter sidebar.

E. Watchlist Management

Add via movie detail page → button state updates optimistically. Dashboard Watchlist tab fetches full list. Remove triggers DELETE request; card disappears via optimistic UI update.

F. FedPCL Training Round (Background)

Bi-weekly cron selects 128 eligible users (is_active, ≥10 interactions). Server dispatches personalized E^{personal} + neighbor embeddings. Client trains 10 local epochs next active session. Client uploads LDP-noised deltas. Server aggregates with FedAvg. K-means recluster every 10 rounds. Validation gate before deployment. All cache keys `user:recommendations:*` flushed on new model deployment.

XXII. Conclusion

Movientum demonstrates that production-quality movie recommendation platforms can combine rich personalization with strong privacy guarantees. The phased architecture enables immediate deployment (Phases 1–4) while accumulating the interaction data necessary for advanced ML (Phase 5). The FedPCL integration provides near-centralized recommendation accuracy ($\geq$60% HR@10 target) without centralizing user data, addressing GDPR compliance through local differential privacy ($\varepsilon = 100$, upgradable to stricter budgets).

The modular monolith design, three-layer backend architecture, and API contract stability across recommendation phases minimize technical debt while enabling seamless ML upgrades. All components — from Nginx rate limiting to K-means cluster reclassification — are documented, parameterized in `params.yaml`/`definitions.yaml`, and instrumented for Prometheus/Grafana observability.

Future work includes: Secure Aggregation for cryptographically masked gradient aggregation, adaptive ε tightening as the model converges, on-device full inference via WebAssembly, asynchronous federated updates removing round synchronization overhead, and LLM-powered semantic search queries.

References

[1] Wang et al., "Personalized Federated Contrastive Learning for Recommendation," *IEEE Transactions on Computational Social Systems*, vol. 12, no. 5, Oct. 2025.

[2] X. He, K. Deng, X. Wang, Y. Li, Y. Zhang, and M. Wang, "LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation," in *Proc. ACM SIGIR*, 2020, pp. 639–648.

[3] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data," in *Proc. AISTATS*, 2017.

[4] S. Rendle, C. Freudenthaler, Z. Gantner, and L. Schmidt-Thieme, "BPR: Bayesian Personalized Ranking from Implicit Feedback," in *Proc. UAI*, 2009, pp. 452–461.

[5] J. C. Duchi, M. I. Jordan, and M. J. Wainwright, "Local Privacy and Statistical Minimax Rates," *Journal of the ACM*, vol. 60, no. 2, 2013.

[6] S. Ramírez, "FastAPI," *GitHub*, 2023. [Online]. Available: https://fastapi.tiangolo.com

[7] The Movie Database, "TMDB API," *themoviedb.org*, 2024. [Online]. Available: https://developer.themoviedb.org